

Software Engineering

Introduction

Building software other people can use

Gregory S. DeLozier, PhD

Today's questions

What is worth building?

What would count as evidence that it works?

How can a team do good work and enjoy doing it?

Engineering: our working definition

Engineering is the application of known principles and the orderly gathering of new knowledge to predictably create a product within the bounds of resource constraints.

Knowledge, problem, constraints

Engineering concern	Question for the team
Science or technology	What do we know? What must we learn?
A real problem	Whose situation should improve?
Constraints	What must fit? What limits our choices?

Software engineering includes operation and maintenance too.

One camera, two reservations

Equipment checkout uses email and a spreadsheet.

Student	Reservation	Reply
Maya	Camera 4, Friday 2-4	Confirmed
Jordan	Camera 4, Friday 3-5	Confirmed

What would we need to know before building anything?

Different causes, different work

What we discover

Staff update separate copies

Equipment comes back late

Students cannot tell if a request was accepted

What might help

A shared reservation record

A return and availability process

Clear confirmation

Find evidence for the cause.

Four weeks to a usable product

Candidate work

Prevent overlapping confirmations

Connect to campus sign-in

Customize calendar colors

Decision to make

Essential for the first release?

What must we learn early?

Can it wait?

What is the smallest useful release?

Make the behavior checkable

Acceptance criterion

A confirmed reservation prevents another student from reserving the same item for an overlapping period.

Where do we still need a decision?

Check the claim

Existing: Camera 4, Friday 2-4, confirmed

New request	Expected result
Camera 4, Friday 3-5	Reject the overlap
Camera 5, Friday 3-5	Allow, if otherwise available
Camera 4, Friday 4-5	Decide the turnaround rule

Two simultaneous requests need a check too.

Show one usable release

1. Reserve Camera 4 and receive confirmation.
2. Try an overlapping reservation.
3. Reopen the application and check the record.

What can a user do with this version?

Work a teammate can continue

Team need	Supporting practice
Understand a change	Version history and review
Find unfinished work	An issue with a clear next step
Run the product	Tested setup instructions

Could a teammate pick this up tomorrow?

A career worth enjoying

Time to concentrate.

Teammates you can rely on.

Work you can finish and take pride in.

How does the team's process help or get in the way?

Same team, same tools

When work becomes exhausting

New priorities; the same deadline

Only one person can fix a release

Decisions stay in private conversations

A change in how we work

Decide what to postpone

Share knowledge and practice recovery

Record decisions where the team works

A new framework will not fix these problems.

Ready for someone else to use?

Can another person use it without us beside them?

Do we know which version is running?

Can we recover from a failed deployment?

Choose the next useful experiment

Unknown

Can our application use campus sign-in?

Small investigation

Check access requirements. Try a minimal connection.

Which decision will the result change?

Who bears the cost of a mistake?

A student arrives for equipment that is unavailable.

A public calendar reveals who borrowed it.

A teammate releases a feature with an unreported failure.

What responsibility follows in each case?

The product will change

Suppose staff decide on a 30-minute turnaround for inspection and charging.

What would need to change?

What evidence would make the change safe to release?

Learning through the project

Make a decision.

Build something that tests it.

Use the result to improve the product and the way we work.

Bring evidence to each project checkpoint.

How the course develops

Course phase	Main work
Understand and plan	Problems, requirements, user stories
Build together	Team workflow, version control, design, testing
Deliver and launch	Deployment, automation, product readiness
Keep it useful	Maintenance, architecture, quality
Judge the consequences	Economics, professional practice, ethics

Start with a problem worth solving

Name one user and one recurring difficulty.

Describe what improvement would look like.

Choose the first thing to observe or ask.

Make the next step small enough to do.